

Model Training Methodology

Preprocessing, Training, Evaluation, and Validation

AI ExamGuard: An AI-Assisted Online Examination Proctoring and Academic Integrity Monitoring System

All figures in this chapter were read directly from the project's training scripts, training-run artifacts, dataset directories, and serialized production models. In-sample, proxy-derived, and negative results are labeled as such where they appear.

1. Overview of the Trained Components

AI ExamGuard runs three **trained** machine-learning components and three **pretrained, unmodified** ones. Only the first group was trained by this project; the second group is used as shipped and is documented here so the boundary between 'what we trained' and 'what we reused' is unambiguous.

Component	Algorithm / architecture	Trained by this project?	Artifact
Phone + face detector	YOLOv8s object detector, fine-tuned from COCO weights	Yes - transfer learning, 8 epochs	app/resources/phone_specialist.pt
Face identity verification	LBPH (Local Binary Patterns Histograms) face recognizer	Yes - one model fitted per enrolled student at enrollment time	backend/storage/<student_id>.yml
Risk scorer	Logistic regression over three windowed detection counts	Yes - fitted on real cheat-event ground truth	app/resources/risk_model.joblib
Face detector	OpenCV YuNet (ONNX), used as published	No - pretrained, unmodified	app/resources/face_detection_yunet_2023mar.onnx
Person counter	YOLOv8s COCO baseline, deliberately never fine-tuned	No - see 4.1 for why it must stay untouched	app/resources/yolov8s.pt
Pose estimator	YOLOv8n-pose, used as published (wrist and nose keypoints only)	No - pretrained, unmodified	app/resources/yolov8n-pose.pt

Table 1. The six model artifacts in the deployed pipeline. Sections 4-6 cover the three trained ones; section 7 covers how F1 was computed; section 8 covers testing and validation.

Head-pose estimation (the PROLONGED_HEAD_DOWN signal) is **not** a trained model at all - it is a solvePnP geometric estimate computed from YuNet's five sparse facial landmarks. It appears in this chapter anyway, in 8.4, because its decision thresholds were calibrated with exactly the same empirical sweep methodology used for the trained models, and because that calibration produced one of the project's more important negative findings.

2. Tools, Libraries, and Compute Environment

Purpose	Tool / library	Notes
Object-detection training and inference	Ultralytics YOLOv8 (PyTorch backend)	YOLOv8s for detection, YOLOv8n-pose for keypoints
Face detection, cropping, recognition	OpenCV (opencv-contrib-python)	cv2.FaceDetectorYN (YuNet) and cv2.face.LBPHFaceRecognizer; the contrib build is mandatory - the plain opencv-python wheel silently removes cv2.face
Risk-model fitting and metrics	scikit-learn	LogisticRegression, roc_auc_score, precision/recall/f1_score

Purpose	Tool / library	Notes
Data handling	pandas, NumPy, Python csv	Window construction, feature tables, bootstrap resampling
Model serialization	joblib (risk model), PyTorch .pt (YOLO), OpenCV .yaml (LBPH)	-
Frame extraction	OpenCV VideoCapture	Webcam-track extraction from the source video corpus
Manual annotation	Labelling (YOLO-format bounding boxes)	Fed with machine-generated draft boxes, then human-corrected
Serving	FastAPI + Uvicorn, containerized with Docker	Inference is deliberately CPU-only in the deployed image

Training compute. The production detector run used a single local CUDA GPU (device: '0' in the run's args.yaml), taking roughly 6,100 seconds of wall-clock time across 8 epochs. The earlier single-class prototype was trained on CPU. **Inference in production is CPU-only by design** - the Docker image has no GPU passthrough - so every threshold sweep and holdout evaluation in this chapter was also run with device="cpu", measuring the deployment configuration rather than a faster one that is never actually served.

3. Data Sources and Preprocessing

3.1 Source corpora

Corpus	Content	Role
MSU Online Exam Proctoring (OEP) database	Real proctored online-exam sessions: 15 'actor' subjects instructed to improvise cheating, 9 'real exam' subjects covertly prompted by a proctor. Ships event-level cheat-type timestamps (gt . txt), no bounding boxes.	Primary real-world source for both the detector (after manual box annotation) and the risk model (directly from gt . txt).
Kaggle samuelayman/cell-phone	Single-class phone bounding boxes.	First-generation prototype detector only; superseded.
Pre-labeled two-class phone/face set (~22.9k images)	Phone and face boxes, already split train/val/test.	Bulk of the production detector's training data.
Locally captured 'back-facing phone' batches	Four sequentially shot real-webcam sessions of the hard low-held / screen-away phone poses.	Targeted hard positives and negatives; also the only data that can evaluate sequence-dependent behavior (8.5).

3.2 Frame extraction from the OEP videos

Each OEP subject folder holds two synchronized video tracks. Only the **webcam** track is extracted; the wearable first-person camera is discarded, because its framing has no relationship to the laptop-webcam viewpoint the deployed system actually receives at inference time. Training on it would optimize for a camera geometry the product never sees.

Extraction is also the project's **anonymization boundary**. The OEP subject folders are anonymized as subjectNN, but the media filenames inside them encode each participant's real username. `extract_oep_frames.py` therefore never propagates a source filename: output frames are named purely from the subject number and timestamp, so no real identity ever enters the derived dataset, the annotation batch, or any artifact in this repository.

3.3 Frame prioritization and batch assembly

One-frame-per-second extraction produced roughly 9,700 raw frames - far more than can be annotated by hand, and heavily redundant. `prioritize_oep_frames.py` classifies every frame against `gt.txt` into *phone*, *other_cheat*, or *clean*, and `build_annotation_batch.py` then assembles a bounded, less redundant annotation folder under a fixed seed:

- **All** phone frames are kept. They are the scarce class, and even near-duplicate frames inside one burst differ in hand and phone position - real variety, not waste.
- *other_cheat* frames are downsampled to roughly twice the phone count, spread **evenly per subject** rather than sampled globally, so one long session cannot dominate the batch.

- *clean* frames are downsampled to roughly 15% of the batch, supplying negatives.
- Original filenames are preserved because they encode subject and timestamp, which is what makes the later leakage-safe per-subject split possible.

3.4 Annotation: machine draft, human ground truth

The OEP corpus has no bounding boxes, so they were created for this project. `auto_annotate_oep.py` opens each frame with **draft** boxes already drawn - YuNet for faces, and the union of the then-current phone specialist plus the COCO *cell phone* class for phones, de-duplicated at IoU 0.5. Draft thresholds are deliberately **looser** than production (phone confidence 0.15 vs 0.35; face score 0.6 vs 0.9), because in a draft-then-correct workflow a missed box costs an annotator far more effort (draw one from scratch) than a spurious one (one click to delete).

Every draft box was then reviewed and corrected by hand in Labellmg. This correction step is not a formality: the entire reason for building the dataset was that the existing detector had blind spots, so training a successor on that detector's own uncorrected output would have taught it to reproduce exactly those blind spots. Frames with a face but no phone were kept as reviewed real negatives; frames with no boxes at all had an **empty label file written explicitly**, so that 'verified as containing nothing' is stored distinctly from 'never labeled'.

3.5 Label-defect remediation

A per-subject accuracy breakdown (8.6) flagged three subjects as far below the overall recall. Visual review of every one of their phone-positive frames found **zero genuine phones**. The boxes were the OEP study's own wired eye-tracking camera clipped to the subject's glasses, background objects, and similar confounders - a labeling defect, not a fairness gap, and one that agreed exactly with `gt.txt`, which documents real phone-use events for only four of the subjects.

Remediation ran in two modes. For the seven subjects with no documented phone use, all phone-class boxes were stripped after a full visual confirmation (a dry run preceded every write). For the four subjects with genuine phone use, all 722 phone-positive frames were reviewed individually rather than blanket-stripped, since those subjects contain a real mixture of held phones and the same eye-tracker confounder; 114 bogus single-box frames were removed. Corrections were applied consistently across the annotation batch, the derived train/val split, and the frozen holdout, so no split kept a stale copy of a corrected label.

One residual defect is recorded rather than papered over: a minority of multi-box frames may still carry a redundant bogus device box alongside the genuine phone box. These were not surgically split, and the consequence is handled on the evaluation side instead - the metric code matches predictions against **all** ground-truth boxes in a frame, not just the first one (7.3).

3.6 Image preprocessing at training and inference

Model	Preprocessing applied
Phone/face detector (YOLOv8s)	Ultralytics letterbox resize to 416x416, RGB, pixel values scaled to [0,1]. Training-time augmentation (mosaic, HSV jitter, horizontal flip, translate, scale, random erasing) is listed in Table 3. No augmentation at inference.
Face recognition (LBPH)	YuNet detects the largest face, the detection box is cropped, converted from BGR to grayscale, and resized to exactly 200x200. That size is mandatory, not cosmetic: LBPH's <code>predict</code> requires the probe to match the enrollment geometry, so an off-spec client crop is re-resized before it reaches the recognizer.
Risk model	No image preprocessing. Its inputs are integer counts produced by the vision models above, aggregated over a sliding time window (6.1).

4. Model A - Phone and Face Detector (YOLOv8s)

4.1 Why a second model instead of fine-tuning the deployed one

The phone detector is trained as a **separate** network that runs as a second inference pass alongside an untouched COCO YOLOv8s. This is a deliberate architectural constraint, not convenience. The phone datasets contain no *person* class, and fine-tuning YOLO on them rebuilds the detection head to emit only the classes present in the new data - which would silently delete the *person* class that the MULTIPLE_PEOPLE violation check depends on. The failure would be silent: the pipeline would keep running and simply stop ever reporting a second person. Keeping the baseline model untouched makes that failure impossible.

4.2 Transfer-learning setup

Training starts from the COCO-pretrained YOLOv8s weights and fine-tunes all layers (no frozen backbone) on the combined two-class dataset. The class ordering is fixed by the dataset's `data.yaml` as `['phone', 'face']`, making class 0 = phone, which is what the serving code asserts. The face class is a free by-product; production face detection still runs through YuNet, not through this model - although the face class did later earn its keep as a false-positive suppressor (4.6).

Hyperparameter	Value	Hyperparameter	Value
Base weights	yolov8s.pt (COCO)	Optimizer	auto (Ultralytics selection)
Epochs	8	Initial LR (lr0)	0.01
Batch size	8	Final LR factor (lrf)	0.01
Image size	416 x 416	Momentum	0.937
Nominal batch size (nbs)	64	Weight decay	0.0005
Early-stop patience	8	Warmup epochs	3
Mixed precision (AMP)	enabled	Random seed	0 (deterministic)
Mosaic augmentation	1.0, closed for last 10 epochs	Horizontal flip	0.5
HSV jitter (h/s/v)	0.015 / 0.7 / 0.4	Translate / scale	0.1 / 0.5
Random erasing	0.4	Vertical flip / shear / perspective	0.0 (disabled)

Table 3. Training configuration of the production detector, transcribed from the run's `args.yaml`. Vertical flip is disabled because a webcam feed is never upside down - allowing it would spend model capacity on an orientation that cannot occur in deployment.

The production run was interrupted by an external event partway through and was **resumed from its own last checkpoint** rather than restarted from epoch 0, which is why the run's recorded `model` field points at its own `last.pt`. The learning-rate schedule and epoch counter continue correctly through a resume, so this does not change the effective training recipe.

4.3 Split strategy: by subject, never by frame

The OEP annotation batch is split into train and validation **by subject**, with a fixed, deliberately chosen validation set rather than a random one. Frames from one subject share the same face, room, camera angle, and lighting, so a per-frame random split would place near-duplicate frames on both sides of the boundary and inflate validation metrics through leakage. The two validation subjects were chosen to cover one phone-heavy 'actor' subject and one 'real exam' subject, so validation retains both genuine phone positives and non-acted footage. The same subject-level discipline is reused for the risk model (6) and generalized to leave-one-subject-out cross-validation there.

Split	Frames	Purpose
OEP annotation batch (after holdout carve-out)	2,260	Source pool for train/val
train	1,837	Gradient updates, combined with the ~22.9k external set
val (2 held-out subjects)	423	Training-time model selection and early stopping
Frozen holdout	322	Final accept/reject evidence only - never trained on, never tuned against

Table 4. OEP frame allocation. The external ~22.9k-image corpus contributes its own publisher-provided train/val split on top of these numbers.

4.4 Training-time metrics

Ultralytics' own per-epoch validation, computed on the combined validation split at the end of each epoch:

Epoch	Precision (B)	Recall (B)	mAP@50	mAP@50-95	val box loss
6	0.727	0.809	0.790	0.509	1.273
7	0.760	0.859	0.818	0.559	1.226
8 (final)	0.769	0.867	0.828	0.566	1.224

Table 5. Final epochs of the production run, from its `results.csv`. These are **aggregate** two-class detector metrics at Ultralytics' own default operating point - they are not the numbers that decided deployment. That decision was made on the frozen holdout, at the app's real threshold, through the full production pipeline (section 7).

Figure reference: `thesis_docs/figures/fig1_training_curves.png`

Figure 1. Per-epoch training curves for the production run (Ultralytics `results.png`). Top row: training box, classification, and distribution-focal losses, plus validation precision and recall. Bottom row: the same three losses measured on validation, plus mAP@50 and mAP@50-95. Two things are worth reading off it. First, **all six loss curves are still descending at epoch 8 and training and validation losses fall together** - there is no divergence between them, so the run shows no overfitting signature and was ended by its epoch budget rather than by convergence or early stopping. Second, **validation recall is visibly noisy from epoch to epoch**, dipping to 0.808 at epoch 6 before returning to 0.867 at epoch 8. That per-epoch wobble is larger than several of the margins that had previously been used to choose between candidate runs, which is a concrete argument for deciding deployment on the frozen holdout (section 8.1) rather than on epoch-level validation metrics.

Provenance check. The deployed `app/resources/phone_specialist.pt` was verified to be byte-identical (matching MD5) to this run's `weights/best.pt`. The metrics and figures in this chapter therefore describe the artifact that is actually serving traffic, not a sibling candidate.

4.5 Per-class performance and error modes

The aggregate mAP in Table 5 averages over two classes that behave quite differently, and the per-class curves matter because production consumes only one of them:

Figure reference: `thesis_docs/figures/fig2_precision_recall.png`

Figure 2. Precision-recall curves per class on the validation split. **The phone class reaches $AP@0.5 = 0.914$, the face class only 0.743**, averaging to the $mAP@0.5$ of 0.828 reported in Table 5. The stronger class is the one that matters: phone detection is what drives a `PHONE_DETECTED` violation, while production face *detection* runs through YuNet and never touches this model. The phone curve holds precision above 0.95 out to roughly 0.85 recall before collapsing, which is the shape a detector needs for this application - room to trade a little recall for high precision. One caveat follows from the weaker curve: the face-shaped false-positive suppressor described in section 4.6 relies on this model's own face-class box, that is, on the less accurate of its two outputs.

Figure reference: `thesis_docs/figures/fig3_confusion_matrix.png`

Figure 3. Column-normalized confusion matrix (columns are ground truth, rows are predictions). **Class confusion is essentially absent** - 0.98 of true phones are predicted phone and 0.96 of true faces predicted face, with only 0.01 leaking either way between the two. The real error mode is the *background* column: among detections that matched no ground-truth box at all, 43% were phone-class and 57% face-class. The background row is the mirror case, showing objects missed entirely - 1% of true phones and 2% of true faces.

Figure 3 also resolves an apparent contradiction with the face-shaped false-positive bug described in the next section. This matrix says the model almost never mistakes a phone for a face - yet the deployed model demonstrably emitted phone boxes at 0.88-0.90 confidence on frames containing only a bare face. Both are true, because they are counted differently: a spurious phone box drawn over a face matches no ground-truth *phone* box, so it is tallied in the background column as a background false positive, not in the face column as a class confusion. The bug is fully present in this matrix; it simply is not visible in the cell one would instinctively check. This is a compact illustration of the chapter's recurring point - an aggregate metric can be accurate and still fail to show the failure that matters.

4.6 The deployed inference pipeline

Evaluating the raw network alone would measure a strictly weaker system than the one deployed. Production runs four stages, and every evaluation script in sections 7 and 8 reproduces all four by importing the same shared code path:

- **Whole-frame pass** at confidence ≥ 0.35 .
- **Face-shaped false-positive suppression.** Any phone box overlapping the model's own same-pass face box at $IoU \geq 0.45$ is discarded. This exists because the detector was found to emit a phone box at 0.88-0.90 confidence on frames showing only a bare face - far above the 0.35 threshold, so it bypassed every downstream safety net. Two attempted retrains failed to fix it (8.7);

this zero-cost geometric post-filter did.

- **Pose-guided hand-crop fallback.** If the whole-frame pass finds nothing, the pose model locates visible wrist keypoints (visibility ≥ 0.10) and the detector re-runs on 320x320 crops centred there, at a deliberately lower threshold of 0.20 - narrowing the search to 'right next to a hand' is itself strong evidence, so a lower score is admissible there than anywhere in frame. The 320 px crop size was swept empirically, not chosen for roundness: raising it from an initial 120 px recovered most of the missed recall, and false positives stayed flat until well past 320.
- **Temporal corroboration.** Detections between 0.20 and 0.35 are held as candidates and only escalate to a logged violation if corroborated across 2 of 3 consecutive polls, which suppresses isolated single-frame noise without lowering the standing threshold.

5. Model B - Face Identity Verification (LBPH)

5.1 Per-student training at enrollment

Unlike the other two models, this one is not trained once and shipped. A **separate LBPH recognizer is fitted per student** at enrollment, from that student's own submitted images:

```
crop = _detect_and_crop(image_bytes)    # YuNet -> grayscale -> 200x200
...
labels = np.array([student_id] * len(samples))
recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.train(samples, labels)
recognizer.write(f"{student_id}.yaml")
```

At least three usable samples are required; images in which YuNet finds no face are dropped, and enrollment is rejected with a corrective message if too few survive. **Raw enrollment photos are never persisted** - only the fitted .yaml model is kept. That is a deliberate privacy design decision, and it has a direct methodological consequence: there exists no stored image set of real enrolled students, so the verification threshold could not be calibrated on real users and required a proxy population instead (5.2).

At verification time the probe frame goes through the identical detect-crop-grayscale-resize path and is scored with `predict`, which returns an LBPH **distance**: lower means more similar. A distance above the threshold is an identity mismatch.

5.2 Threshold calibration on a proxy population

The original threshold of 80.0 was an unvalidated default. It was calibrated by treating the OEP subjects - real people, real webcams, real lighting variety, recorded under similar shared study conditions - as a stand-in population. For each subject, a handful of their frames train an LBPH model exactly the way enrollment does; held-out frames from the **same** subject become genuine attempts, and frames from **every other** subject become impostor attempts. This is a plausible worst-case impostor model for this application: a classmate under similar camera and lighting conditions, not a random stranger.

Genuine and impostor distance distributions overlap substantially (genuine median 42.2, ranging up to 90.0; impostor starting at 48.4, median 80.6):

Threshold	Precision	Recall	False Accept Rate	False Reject Rate	Verdict
80.0 (original default)	0.552	0.979	47.6%	~2%	Rejected - nearly half of impostor attempts accepted, defeating the check's entire purpose
60.0 (deployed)	0.950	0.873	2.7%	12.7%	F1-optimal on this data; adopted

Table 6. Face-verification threshold sweep. The trade is explicit: legitimate students are false-flagged on roughly 1 check in 8 instead of 1 in 48, in exchange for closing a 47.6% false-accept hole. This is defensible only because a flag is reviewable evidence for an instructor, not an automatic sanction.

Offline-to-live transfer was then confirmed, not assumed. Five genuine trials on a real enrolled student with a real webcam scored 40.8-44.2, closely matching the offline genuine median of 42.2. This step exists because an earlier offline sweep for the phone detector transferred badly to live hardware (8.3), and the lesson - verify a swept threshold against live capture before trusting it - was applied here.

5.3 A ceiling found, and honestly recorded

The same live test surfaced a problem that no threshold can fix: **a photograph of a different person held up to the webcam scored 41.9-46.2** - squarely inside the genuine range. LBPH compares texture and has no concept of liveness, so this is an anti-spoofing gap rather than a calibration gap.

The first countermeasure is a frame-difference liveness check: a live face exhibits continuous micro-movement between polls, a static photo does not. Live measurement produced a partial result stated here in full:

- **Genuine, deliberately still** (reading a question, trying not to move): consecutive-poll mean absolute difference 11.3-30.9.
- **Hand-held photo** (the realistic attack): 7.2-20.3 - **overlapping** genuine. Natural hand tremor moves the photo enough to imitate a living person. No threshold separates these two distributions; this ceiling stands.
- **Rigidly propped photo, no hand contact**: 3.06-6.68, pooled over two independent capture rounds, with zero overlap against genuine's 11.3 floor.

The threshold was accordingly raised from 3.0 to 8.0 - above the rigid-spoof maximum of 6.68, safely below the genuine floor of 11.3 - which closes the rigid-mount sub-case only. The hand-held case remains open and is documented in the source as a signal ceiling, with the note that closing it would require a categorically different signal (such as blink detection from dense eyelid landmarks, which no photograph can produce) rather than further tuning of this one.

6. Model C - Risk Scorer (Logistic Regression)

6.1 Feature construction

The risk model replaces what were previously hand-guessed weights for the three vision signals. Its training data is built in two deliberately separated stages, because detection is slow (about 1 s per poll on CPU across three YOLO passes plus YuNet) while the labeling rule is a judgment call worth iterating on cheaply:

- **Stage 1 - cache detections.** The real production detectors are run over the extracted OEP webcam frames, sampled at 15-second intervals to match the live polling cadence exactly, and the raw per-poll outcomes (face lost, phone detected, person count) are cached to disk. This produced 1,546 cached polls.
- **Stage 2 - window and label.** A trailing 120-second window - the same WINDOW_SECONDS the live risk service uses - slides over each subject's poll sequence, counting how many polls inside it flagged each event type. This mirrors exactly what the production risk service counts from real violation rows, so the model is trained on the same feature distribution it is served.

The three features are therefore `face_lost_count`, `phone_detected_count`, and `multiple_people_count`. Browser-behavior signals (tab switching, copy-paste, fullscreen exit) are excluded from the model and remain hand-weighted, because no video ground truth exists to train them against - they are not visible in webcam footage at all.

6.2 The labeling rule, and what was deliberately excluded

The OEP ground truth defines five cheat types. Only some have any correlate a webcam-based vision system could plausibly observe, and the labeling rule says so explicitly:

Cheat code	Meaning	Treatment
4, 5	Phone call, phone use	Positive - directly what phone detection targets
2	Talking to someone	Positive - the documented protocol had a proctor talk to, walk up to, or hand a book to the student, so a second body plausibly enters frame
1, 3, 6	Reading notes, internet use, undefined	Excluded from the dataset entirely - no plausible visual correlate. Reading notes does not reliably move the face out of frame; internet use is invisible to a webcam

Table 7. Vision-plausible label mapping. A window whose only overlapping cheat code is 1, 3, or 6 is dropped rather than forced into either class.

This exclusion was not a stylistic preference - it was forced by a real failure. The first version labeled *any* overlapping cheat code as positive, which meant the two validation subjects (one almost entirely 'talking', one mostly 'reading notes') were tested nearly exclusively on cheat types with no visual signal at all. The model scored AUC 0.51 - indistinguishable from chance. The fix was restricting labels to what the sensors can actually observe, not collecting more data: codes 1, 3, and 6 are structurally

invisible no matter how many subjects are added.

6.3 Fitting

Logistic regression over the three counts, with `class_weight="balanced"` and `C=1.0`. The choice of a linear, inspectable model over something more powerful is deliberate and is the point: the learned coefficients preserve exactly the interpretability the original hand-weighted formula had - 'how much does each signal move the score' - while being fitted from real cheat-event ground truth instead of guessed. In a system whose outputs are contestable by students through an appeals workflow, a coefficient a human can read out and defend is worth more than a marginal accuracy gain from an opaque model.

The production model is fitted on **all** subjects rather than on a train/val split. That is safe only because leave-one-subject-out cross-validation (8.2) already supplies an honest generalization estimate without permanently reserving data; the training script prints its in-sample fit under an explicit label stating it is not a held-out metric and pointing at the cross-validation number instead.

Property	Value
Windows / subjects	1,107 windows across 24 subjects
Positive rate	71.0%
face_lost_count coefficient	+0.012 log-odds per unit count
phone_detected_count coefficient	+0.027 log-odds per unit count
multiple_people_count coefficient	+1.710 log-odds per unit count
Intercept	-1.511

Table 8. Fitted production risk model, read from the serialized `risk_model.joblib`. The coefficients are themselves a finding: a second person in frame dominates the score by roughly two orders of magnitude over the other two signals, which is consistent with 'a second body is unambiguous, while a lost face or a borderline phone box is frequently benign'.

6.4 Output rescaling

The OEP subjects were scripted or prompted to cheat, so the training set's 71% positive rate is nowhere near a real exam population's. That inflated base rate lives in the intercept, and its consequence is visible: the raw model assigns roughly 0.16 probability to a session with **zero** detected violations. A clean session must score 0, not an unexplained baseline, so the serving layer rescales the raw probability against that zero-evidence floor - the floor maps to 0, and the already near-1 maximum-evidence case stays near 1. This changes only the output scale; it does not alter what the model learned about how the three signals relate.

7. F1 Score: Definition and Computation

7.1 The formulas

Precision, recall, and F1 are computed from raw counts of true positives (TP), false positives (FP), and false negatives (FN), with F1 as their harmonic mean:

```
precision = TP / (TP + FP)
recall    = TP / (TP + FN)      # equivalently TP / total_positives
F1        = 2 * precision * recall / (precision + recall)
```

The harmonic mean is the appropriate summary here precisely because it refuses to be rescued by one strong component: a detector that flags every frame reaches recall 1.0, and a detector that flags nothing but its single most confident frame can reach precision 1.0. Both score near zero on F1. For proctoring, both failure modes are unacceptable in different ways - the first buries instructors in false accusations, the second silently misses real ones - so the metric must penalize each.

7.2 The unit of evaluation is a frame, not a box

This is the single most important definitional choice in the chapter, and it is dictated by what the deployed system actually does. Production's real check is a **presence** test - 'did any phone-class detection survive in this frame' - which then drives whether a violation is logged. It has no location awareness whatsoever. So the metric counts frames:

- **TP** - a frame that contains a real phone and was flagged.
- **FP** - a frame that contains no phone and was flagged.
- **FN** - a frame that contains a real phone and was not flagged.
- A frame with no phone and no flag is a true negative, and correctly contributes to none of precision, recall, or F1.

An earlier version of the evaluation required an IoU match against the ground-truth box before counting a detection as correct. That was a stricter and **different** question from the one production asks, and it silently mis-counted: a correct-verdict-but-wrong-location detection on a phone-positive frame was recorded only as a missed true positive, never as the false positive it simultaneously was. Both metrics are now computed and reported side by side:

Metric	Definition	Status
Presence	Any surviving phone-class detection anywhere in frame, from the whole-frame pass or the hand-crop fallback, counts as a flag. Reproduces <code>object_detection_service.py</code> 's deployed check exactly.	The headline number. Every accept/reject decision cites this.
Localized	Additionally requires the detection to overlap a real ground-truth box at $\text{IoU} \geq 0.3$.	Diagnostic only. Catches 'right answer for the wrong reason' - it is what exposed the eye-tracker-device confounder described in 3.5.

7.3 Two evaluation bugs that changed the numbers

Both were found and fixed before any reported figure was produced, and both are recorded because each had been silently distorting results:

- **The evaluation omitted the hand-crop fallback.** The scripts scored only the whole-frame pass, that is, a strictly weaker pipeline than the one deployed, so real recall was being *understated*. Both the holdout evaluation and the threshold sweep now import one shared module that makes the identical model calls production makes - the fix was to unify the code path, not to keep two implementations in sync by discipline.
- **Ground-truth matching read only the first box in a frame.** Frames legitimately carry more than one phone-class box (3.5), so IoU was sometimes being computed against the wrong box - occasionally the residual bogus one. This alone manufactured an apparent per-subject recall gap that was later shown to be an evaluation defect, not a model or fairness defect (8.6). The loader now returns every box in the frame, and a prediction matching any of them counts.

7.4 Worked computation - the deployed detector

Evaluated on the 322-frame frozen holdout (96 phone-positive, 226 phone-negative), through the full production pipeline, at production's real 0.35 confidence threshold:

```
TP = 76,  FP = 6,  FN = 96 - 76 = 20

precision = 76 / (76 + 6) = 76 / 82 = 0.927
recall    = 76 / 96      = 0.792
F1        = 2 * 0.927 * 0.792 / (0.927 + 0.792) = 0.854
```

Read plainly: of every 100 frames the system flags for a phone, about 93 really contain one; of every 100 frames that really contain a phone, about 79 are flagged. The asymmetry is intentional. In proctoring a false accusation is more costly than a missed frame, and a missed *frame* is not a missed *event* - phone use spans many consecutive polls, so the system has repeated independent chances to catch a single real episode, while each false positive is a fresh burden on a real student.

7.5 F1 for the other two models

Risk model. Fitted through scikit-learn, so precision, recall, F1, and ROC-AUC come from `sklearn.metrics` on the same TP/FP/FN definitions, with a window (not a frame) as the unit and the default 0.5 decision threshold for the thresholded metrics. The training script also evaluates the **old hand-set weights** on the identical validation windows, so the improvement from training is measured against the thing it replaced rather than assumed. Because a threshold-free summary is more informative for a scorer whose output is a continuous 0-100 value, ROC-AUC - not F1 - is the headline metric for this model (8.2).

Face verification. Reported as precision and recall plus False Accept Rate and False Reject Rate (Table 6), since FAR and FRR are the biometric-standard framing and speak directly to the two asymmetric costs: an accepted impostor defeats the check, a rejected genuine student is a wrongly flagged innocent person.

7.6 The training-time F1 curve, and why it is not the headline number

The training run produces its own F1 curve, and it is included here specifically because it is easy to mistake for the number in 7.4. It is not the same measurement:

Figure reference: `thesis_docs/figures/fig4_f1_confidence.png`

Figure 4. F1 against confidence threshold on the validation split. All classes peak at **F1 = 0.80 at confidence 0.433**; the phone class alone peaks near 0.88. **This is not the 0.854 reported in section 7.4** - this curve is box-level and IoU-matched, computed per detection on the validation subjects, whereas 0.854 is frame-level presence on the frozen holdout through the full four-stage production pipeline. The two answer different questions and are not interchangeable.

Two things are genuinely useful in Figure 4. First, **the curve is broad and flat between roughly 0.2 and 0.6** - across that whole band F1 barely moves. Production's threshold of 0.35 sits comfortably inside that plateau, so the operating point is insensitive to small changes there, which is a reassuring property for a threshold that must survive varying real lighting.

Second, and more instructive: **this curve's optimum (0.433) disagrees with the frozen-holdout sweep's recommendation of 0.60-0.70 (8.3)**. Two offline analyses of the same model, on different data through different pipelines, pointed at materially different thresholds - and the higher of the two, when actually deployed, collapsed live recall to roughly 4%. That disagreement is the argument for the rule this project now follows: an offline optimum is a hypothesis to be checked against live capture, not a decision.

8. Testing and Validation

8.1 The frozen holdout, and the leakage it exists to prevent

The two validation subjects are a legitimate held-out set for training-time model selection. But they had also been reused, run after run, to decide whether each new candidate should replace production. Repeatedly consulting one set to make a *sequence* of accept/reject decisions is test-set hill-climbing, not evidence of generalization. Worse, several of those decisions had also quoted accuracy on live-capture batches the candidate had just been trained on - a training-set number reported alongside a genuinely held-out one without a label distinguishing them.

The remedy is a **frozen holdout**: a stratified 15% slice (322 frames, fixed seed) drawn proportionally from every train-eligible source - each OEP subject and each live-capture batch - and physically **moved out** of the annotation folder, so the split script cannot sweep it back into training even by accident. Enforcement is structural rather than procedural. Since all four subjects with real phone use were already committed to train or val, a stratified slice was the honest best available option, not a first-choice design; the constraint is recorded as such.

The usage rules are written into the scripts themselves: never train on it; never use it to pick a checkpoint, tune a threshold, or choose between mid-development candidates; evaluate once per genuine swap candidate; report the result even when unflattering. The script that carves the holdout refuses to re-carve an existing one without an explicit force flag, because re-sampling after evaluation would reintroduce exactly the leakage it exists to prevent.

8.2 Leave-one-subject-out cross-validation (risk model)

The risk model's original headline AUC of 0.879 was a single point estimate from two held-out subjects - and those two had been inherited from the detector's split by convention, not chosen for this purpose. Too thin to support a generalization claim, especially given that a different pair had once produced AUC 0.51 by accident of which cheat types they happened to contain.

Leave-one-subject-out cross-validation refits the model once per subject, each time holding out one **whole subject**. A held-out window from a subject whose other windows are in training tests almost nothing new - the subject is the real unit of generalization risk, because it carries the camera, the lighting, and the individual's pose habits. Three things are reported:

- **Per-subject held-out AUC** where computable. Subjects with zero positive or zero negative windows are reported as *n/a* with the reason - never silently skipped, never scored 0.
- **Pooled LOSO AUC** - all held-out predictions concatenated, one AUC over all of them. This is the headline generalization number and the fair comparison against the old single-split 0.879.
- **Subject-level bootstrap 95% CI** - resampling **which subjects** contribute, with replacement. Resampling individual windows would be wrong: windows within a subject are correlated (same room, same person), so window-level resampling would badly understate the real uncertainty. Degenerate all-one-class resamples are skipped and counted, not scored.

Dataset	Subjects	Windows	Pooled LOSO AUC	Subject-level bootstrap 95% CI
Original	11	-	0.797	[0.666, 0.941] (width 0.275)
Expanded (production)	24	1,107	0.813	[0.712, 0.902] (width 0.190)

Table 9. The dataset grew when 13 further real subjects with usable ground truth were found already on disk, unextracted. The interval narrowed by roughly 31% - a genuine tightening of the uncertainty, which is the more meaningful half of this result; the AUC itself moved only 0.016. The interval remains wide, and that is reported as a real limitation, not smoothed over.

8.3 Threshold sweeping, and a live regression it caused

Operating thresholds were swept rather than guessed. The sweep runs the full pipeline **once** per holdout image at a low confidence floor, recording every candidate score, and then sweeps the threshold analytically over those recorded scores - so the holdout is inspected once, not once per candidate threshold.

That sweep recommended raising the phone threshold from 0.35 to a much higher value, on the basis of a clear offline F1 improvement. **It was deployed and it failed.** A live browser smoke test immediately afterwards, on a genuine continuously-visible phone, found real recall had collapsed to roughly 4%. Diagnosis: the holdout never sampled the lighting conditions of the live capture setup, so live confidence scores sat systematically below the offline ones and fell under the raised bar. The threshold was reverted to 0.35.

This is retained in the methodology as a load-bearing finding rather than an embarrassment. It establishes the rule the project now follows everywhere - **an offline aggregate metric can be clean and still be wrong about live behavior** - and it is the direct reason the face threshold in 5.2 was live-verified before adoption, and the reason the sequence-replay evaluation in 8.5 exists at all.

8.4 Threshold validation for head-down detection

The head-down thresholds were validated with the same sweep methodology, using human-labeled phone-present frames as a real-world proxy for 'looking down'. The result is a candid negative one:

Threshold	Value	Precision	Recall	F1	Assessment
Head pitch	-35 deg	0.50	0.57	0.53	F1 peak - no threshold does meaningfully better. Pitch alone is a genuinely weak signal; this is a ceiling, not a mistuned number
Sustained duration	25 s	-	-	-	Matches real sustained-episode timing measured from densely timestamped label sequences; ignores brief blips
Pose-fallback confidence	0.10	0.944	1.000	0.971	Never once misses a real head in this data; raising it trades that perfect recall for marginal precision - a bad deal here

Table 10. Head-down threshold validation.

Roughly 50% frame-level precision is documented in the source as a real limitation of a pitch-only signal, and it is the reason this signal is never sufficient on its own: the duration requirement, the captured question context shown to the reviewing instructor, and mandatory human review all sit downstream of it. The same analysis also found and fixed a real bug - a single noisy poll was resetting an accumulating head-down episode, discarding a genuine sustained event, which is why a one-poll miss tolerance now exists.

8.5 Sequence replay for time-dependent behavior

Temporal corroboration cannot be evaluated by any script that treats frames independently or shuffles them - the feature is defined over consecutive polls. A dedicated evaluation replays each live-capture batch **in its original filming order**, which is the order a live polling loop would have observed, and reports old single-frame behavior against new corroborated behavior per batch, naming the specific frames corroboration recovered and the ones it broke. It reuses the same shared analysis function, so it measures the deployed pipeline rather than a reimplementations of it.

8.6 Fairness sensitivity audit

A tier-1 audit reports per-subject and per-capture-batch accuracy across all real labeled frames, flagging any group more than 15 points below the overall figure. Its limitation is stated first and not buried: **the source corpus carries no demographic metadata** - no skin tone, gender, age, or camera-quality labels. The audit can say 'this subject scores lower'; it cannot say why. Every disparity it surfaces is a lead for a real demographic audit (recruit deliberately, label attributes, re-run), never a finished fairness conclusion.

It has nonetheless earned its place twice. It found the label defect described in 3.5. And when it flagged one subject at 0.741 recall against 0.918 overall, re-measuring after the multi-box ground-truth fix (7.3) showed **no disparity at all**: 0.898 presence recall against 0.948 overall across 2,582 frames, with the audit's own disparity flag returning nothing. The gap had been an evaluation defect. The procedural rule taken from this - visually review a flagged group's actual frames before concluding the model is at fault - is recorded for reuse.

Note that this audit deliberately draws from **both** the training pool and the frozen holdout, unlike every other evaluation here. It is a diagnostic characterization of the deployed model rather than a swap decision, so per-group sample size (statistical power to detect a disparity at all) matters more than avoiding training contamination. Groups drawn from training data are labeled as such in its output so the two are never conflated.

8.7 Negative results

Three are recorded, because a methodology that only reports what worked is not a methodology:

- **Two retrains on additional hand-corrected data both regressed.** A 1,773-frame corrected personal-video batch was merged into training twice - once in full, once downweighted. Frozen

holdout F1 came out at 0.833 and 0.838 against production's 0.854. The batch was dropped from the training set entirely and production weights were never touched. The underlying false-positive bug was later fixed by the zero-cost geometric post-filter in 4.6 - **more data was the wrong tool for that problem.**

- **A second independent pitch signal from the pose model was tried and reverted**, on measured evidence of worse F1.
- **The offline threshold sweep of 8.3 was deployed and reverted** after live measurement contradicted it.

8.8 Software-level regression testing

Model behavior is only half of correctness. The backend carries **109 automated tests**, all passing at the most recent full run, covering authentication, the permission model, schema integrity, violation handling, head-down tracking, head-pose estimation, and an end-to-end system smoke test. Detection services expose synthetic-input paths so the surrounding logic can be tested without loading model weights, and the pure-bookkeeping parts of the pipeline (the static-image streak counter, the corroboration candidate window) are deliberately factored out of the model-calling code specifically so they remain testable in isolation.

9. Limitations

Stated plainly, because each one bounds how the numbers in this chapter may legitimately be read:

- **The subject population is small and not demographically characterized.** 24 real subjects, no skin-tone, gender, age, or camera-quality metadata. The fairness audit is a sensitivity analysis over incidental variation, not a demographic audit.
- **The risk model's confidence interval is wide.** Pooled LOSO AUC 0.813, 95% CI [0.712, 0.902]. The point estimate should not be quoted without the interval.
- **Risk training labels come from a scripted cheating population** with a 71% positive rate, far from any real exam. The output rescaling in 6.4 corrects the score's zero point; it cannot correct the underlying prevalence mismatch.
- **Face-verification calibration used a proxy population**, because real enrolled students' photos are deliberately never stored. Live testing confirmed the genuine distribution transfers, but the impostor distribution was never validated against real impostor attempts.
- **The hand-held photo spoof is not caught** by the current liveness check, and this is a signal ceiling rather than a tuning gap (5.3).
- **Head-pitch precision is roughly 50% at the frame level** - the signal requires duration, question context, and human review to be usable at all (8.4).
- **A minority of multi-box training frames may retain a redundant spurious box** (3.5). The evaluation code compensates; the training labels still carry it.
- **Offline metrics have been shown to mispredict live behavior on this system** (8.3). Every reported figure is offline unless it says otherwise.

10. Reproduction

The pipeline is reproducible end to end from this repository. All scripts live in `backend/training/`; the external image corpora must be obtained separately and their paths set in the corresponding `data.yaml`. Seeds are fixed throughout (dataset assembly, holdout carving, bootstrap resampling, and the detector run itself at `seed: 0, deterministic: true`).

```
# Detector
python extract_oep_frames.py           # webcam track only, anonymized filenames
python prioritize_oep_frames.py        # classify frames against gt.txt
python build_annotation_batch.py       # bounded, per-subject-balanced batch
python auto_annotate_oep.py           # DRAFT boxes -> hand-corrected in LabelImg
python make_frozen_holdout.py          # carve the frozen holdout FIRST
python prepare_oep_split.py            # subject-level train/val split
python finetune_phone_face.py --epochs 8 --batch 8 --imgsz 416

# Risk model
python extract_risk_polls.py           # real detectors at the 15s live cadence
python build_risk_windows.py           # 120s windows, vision-plausible labels only
```

```
python loso_cv_risk_model.py          # generalization estimate + bootstrap CI
python train_risk_model.py --final    # production fit on all subjects

# Evaluation and validation
python evaluate_frozen_holdout.py --model ../app/resources/phone_specialist.pt
python threshold_sweep.py            # analytic sweep from a single inference pass
python evaluate_corroboration.py     # sequence replay in original filming order
python fairness_audit.py             # per-subject / per-batch disparity check
python analyze_face_recognition_threshold.py
python analyze_head_pose_thresholds.py
python analyze_pose_fallback_threshold.py
```

One ordering constraint is not optional: the frozen holdout must be carved before the train/val split is generated. Reversing the order puts holdout frames into training and invalidates every subsequent evaluation number in this chapter.